PROJETO – Eng.Sw.2 2026.2

ClinicalStudyPB — Sistema de Gestão de Estudos Clínicos
Proposta de projeto para Engenharia de Software II

1. Escopo geral

O ClinicalStudyPB é um sistema simplificado para gerenciar o ciclo de vida operacional de estudos clínicos, desde o cadastro de um protocolo de pesquisa até o acompanhamento dos participantes e encerramento do estudo.
O sistema não realiza diagnósticos nem decisões clínicas automatizadas. Seu objetivo é apoiar a organização, execução e acompanhamento de estudos clínicos, utilizando dados fictícios.



O ClinicalStudyPB deverá permitir o gerenciamento simplificado de estudos clínicos, abrangendo o cadastro de protocolos, recrutamento, elegibilidade, inclusão de participantes, agendamento de visitas, acompanhamento e relatórios.

2. Perfis de usuários

Participante
Consulta suas visitas agendadas, acompanha sua participação no estudo, registra informações solicitadas e pode solicitar a retirada do estudo.

Pesquisador
Cadastra protocolos, define critérios de elegibilidade, acompanha participantes, consulta dados e gera relatórios.

Coordenador do Estudo
Gerencia recrutamento, verifica elegibilidade, organiza visitas, acompanha pendências e monitora o andamento do estudo.

Administrador
Gerencia usuários, perfis de acesso, configurações e registros de auditoria.

Fluxo geral do sistema



3. Requisitos Funcionais

Módulo A — Usuários e autenticação

ID	Requisito	Interface
RF01	Cadastrar e gerenciar usuários com perfis de pesquisador, coordenador, administrador e participante.	Web
RF02	Permitir autenticação e encerramento de sessão.	Web/Mobile
RF03	Restringir funcionalidades conforme o perfil do usuário.	Web/Mobile

Módulo B — Gestão de estudos

ID	Requisito	Interface
RF04	Cadastrar estudos com título, objetivo, responsável, datas e limite de participantes.	Web

RF05	Definir critérios de inclusão e exclusão para cada estudo.	Web
RF06	Definir visitas previstas e suas atividades.	Web
RF07	Gerenciar os estados do estudo: planejamento, recrutamento, execução e encerrado.	Web

Módulo C — Recrutamento

ID	Requisito	Interface
RF08	Cadastrar candidatos vinculados a um estudo.	Web
RF09	Registrar a avaliação dos critérios de elegibilidade.	Web
RF10	Registrar o consentimento simulado do candidato.	Web
RF11	Incluir candidatos elegíveis e com consentimento registrado, respeitando o limite de vagas.	Web

Módulo D — Acompanhamento

ID	Requisito	Interface
RF12	Agendar visitas para os participantes incluídos.	Web
RF13	Registrar a realização, ausência ou cancelamento de visitas.	Web
RF14	Registrar resultados fictícios das avaliações previstas.	Web
RF15	Consultar o histórico de visitas e avaliações de um participante.	Web
RF16	Registrar a retirada de um participante do estudo.	Web

Módulo E — Aplicativo do participante

ID	Requisito	Interface
RF17	Permitir ao participante consultar os estudos em que está incluído.	Mobile
RF18	Exibir as próximas visitas e seu histórico de comparecimento.	Mobile
RF19	Permitir confirmar presença prevista ou solicitar reagendamento de visita.	Mobile
RF20	Permitir solicitar retirada voluntária do estudo.	Mobile

Módulo F — Monitoramento

ID	Requisito	Interface
RF21	Exibir painel com quantidade de candidatos, participantes incluídos e visitas realizadas ou pendentes.	Web
RF22	Gerar relatório simplificado de recrutamento e acompanhamento por estudo.	Web

Requisito surpresa: será definido pela professora durante a Release 3.



4. Organização da equipe — 6 integrantes

Papel	Responsabilidades
1. Líder técnico / Gerente	Arquitetura, API, integração e decisões técnicas
2. Desenvolvedor Backend	Regras de negócio, persistência e testes
3. Desenvolvedor Web	Telas de estudos, protocolos e usuários
4. Desenvolvedor Web	Telas de recrutamento, visitas e relatórios
5. Desenvolvedor Mobile	Aplicativo do participante
6. QA / DevOps	Testes, integração contínua, qualidade e suporte à implementação

A equipe deverá compartilhar também as responsabilidades de planejamento, documentação e acompanhamento das sprints. O papel de QA não deve significar que apenas um integrante desenvolva testes.

5. Cronograma de sprints

- SPRINT1: 13/10 - 26/10
- SPRINT2: 27/10 - 09/11
- SPRINT3: 10/11 - 23/11
- SPRINT4: 24/11 - 07/12
- SPRINT5: 08/12 - 17/12
- SPRINT6: 02/02 - 11/02

Release 1 — Estrutura e recrutamento

09/11/2026

Sprints 1 e 2

Sprint 1 (13/10 a 26/10)

- Configuração do repositório e arquitetura.
- Estrutura inicial de backend, web e mobile.
- RF01 a RF04: usuários, autenticação, autorização e cadastro de estudos.
- Protótipos navegáveis das interfaces.

Sprint 2 (27/10 a 09/11)

- RF05 a RF11: critérios, visitas previstas, estados, recrutamento, elegibilidade e inclusão.
- Integração das telas web com o backend.
- Testes e correção de defeitos.

Entrega: aplicação web funcional permitindo cadastrar um estudo, avaliar candidatos e incluir participantes.

Release 2 — Acompanhamento e mobile

07/12/2026

Sprints 3 e 4

Sprint 3 (10/11 a 23/11)

- RF12 a RF16: agendamento, acompanhamento e retirada de participantes.
- Implementação do login mobile.
- RF17 e RF18: estudos e visitas no aplicativo.

Sprint 4 (24/11 a 07/12)

- RF19 e RF20: confirmação, reagendamento e retirada pelo mobile.
- RF21 e RF22: painel de acompanhamento e relatórios.
- Integração completa entre web, mobile e backend.
- Testes integrados.

Entrega: sistema com interfaces web e mobile funcionais e fluxo de acompanhamento completo.

Release 3 — Manutenção evolutiva

11/02/2027

Sprints 5 e 6

Sprint 5 (08/12 a 17/12)

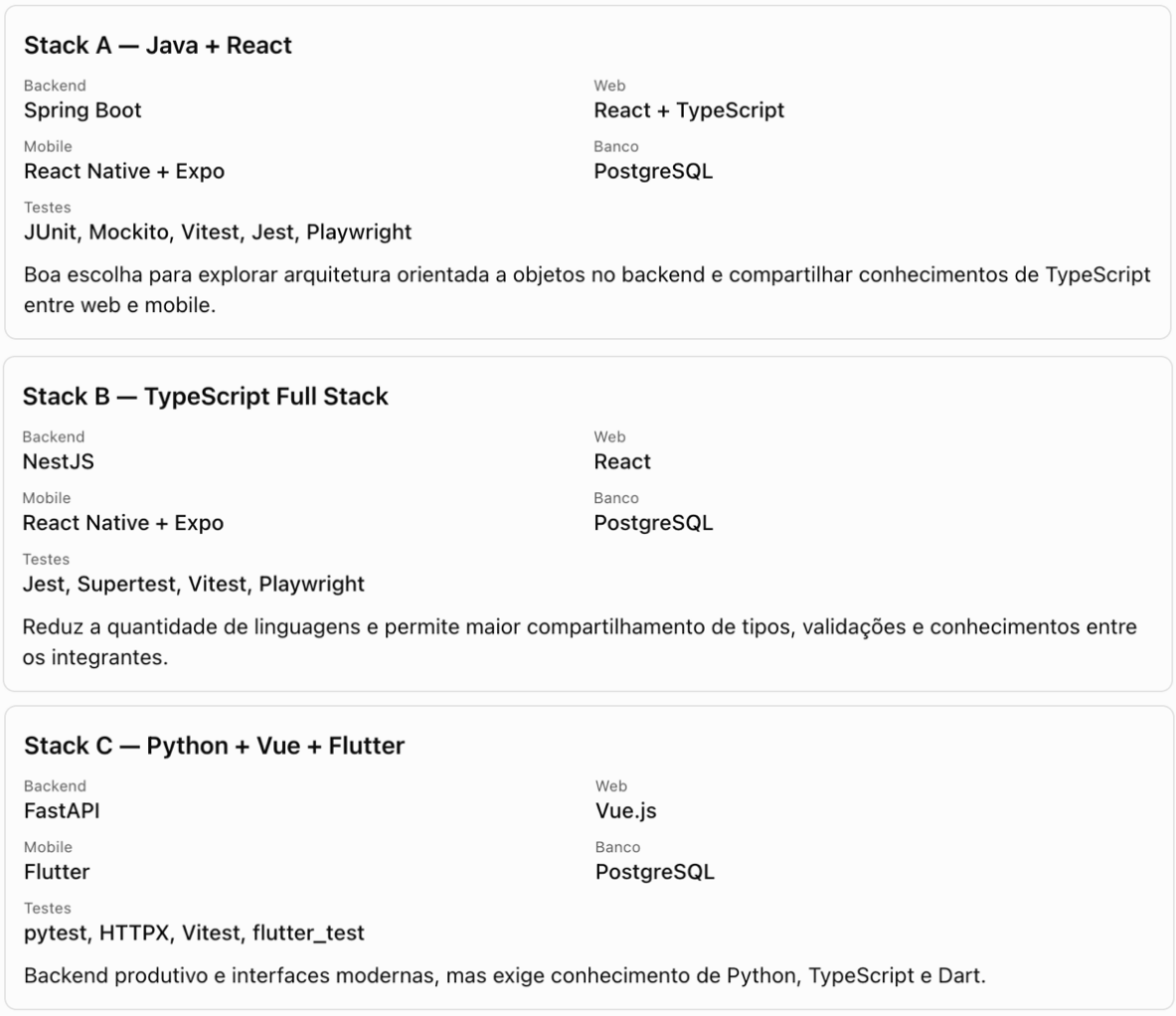
- Troca de projetos entre equipes.
- Análise da arquitetura e do código recebido.
- Execução dos testes existentes.
- Identificação de problemas e elaboração do plano de manutenção.
- Definição dos requisitos surpresa.

Sprint 6 (02/02 a 11/02)

- Implementação dos novos requisitos.
- Refatoração de componentes necessários.
- Testes de regressão.
- Atualização dos diagramas e da documentação.
- Apresentação final.

Entrega: versão evoluída de um sistema desenvolvido por outra equipe.

6. Stack Tecnológico



***Para o mobile, é suficiente uma aplicação executável em emulador ou dispositivo. Não é necessário publicar nas lojas de aplicativos.

7. Estratégia de testes

Como Engenharia de Software II enfatiza qualidade de software, recomendo exigir testes em diferentes níveis, independentemente da tecnologia escolhida.

Nível	O que testar	Exemplos
Unitários	Regras de negócio isoladas	Elegibilidade, limite de participantes, estados do estudo
Integração	Comunicação entre componentes	API, serviços, repositórios e banco de dados

Componentes web	Comportamento das telas	Validação de formulários, botões, mensagens de erro
Componentes mobile	Comportamento das telas mobile	Login, listagem de visitas, confirmação
Ponta a ponta (E2E)	Fluxos completos	Cadastrar estudo, incluir participante e agendar visita

Critérios mínimos sugeridos

- Cobertura mínima de 80% das linhas do código de negócio do backend, excluindo código gerado e configurações.
- Testes unitários para todas as regras de negócio relevantes.
- Testes de integração para os principais endpoints.
- Testes automatizados de componentes nas interfaces web e mobile.
- Pelo menos um fluxo E2E automatizado no sistema web e um no aplicativo mobile.
- Execução automatizada dos testes a cada pull request ou integração à branch principal.

8. Ferramentas complementares

Finalidade	**Opções**
Versionamento	GitHub, GitLab
Gerenciamento ágil	Azure DevOps
Prototipação	Figma, Penpot
Documentação da API	OpenAPI/Swagger
Teste de API	Postman, Insomnia, Cypress
Qualidade de código	SonarQube, SonarCloud, ESLint, Checkstyle
Cobertura de testes	JaCoCo, Istanbul, Coverage.py
Integração contínua	GitHub Actions, GitLab CI
Containers	Docker, Docker Compose
Diagramas	PlantUML, Mermaid, diagrams.net

9. Como definir as tecnologias para o projeto

Liberdade tecnológica controlada.

Proposta de regras para escolha tecnológica

1. Cada equipe poderá escolher livremente entre os stacks tecnológicos sugeridos.
2. O backend deverá disponibilizar uma API HTTP REST compartilhada pelas interfaces web e mobile. A API deverá ser documentada no padrão OpenAPI, podendo utilizar Swagger UI para consulta e testes dos endpoints.
3. A interface web deverá atender aos perfis de pesquisador, coordenador e administrador.
4. A interface mobile deverá atender ao perfil de participante e executar em dispositivo ou emulador.
5. O sistema deverá possuir persistência de dados e controle de acesso por perfil.
6. As equipes deverão implementar testes automatizados e apresentar evidências de cobertura.
7. O projeto deverá conter documentação de instalação e execução, permitindo que outra equipe assuma o desenvolvimento na Release 3.
8. As tecnologias escolhidas deverão ser documentadas e justificadas na Sprint

10. Sobre as entregas

Cada release deverá ser entregue por meio do **repositório da equipe no GitHub**, contendo o código-fonte e a documentação correspondente àquela versão. A documentação será organizada no **README.md**, substituindo os relatórios em PDF utilizados nos projetos anteriores.

Implementação no repositório

O repositório deverá conter:

- Código-fonte do backend, da aplicação web e do aplicativo mobile, conforme os requisitos previstos para a release.
- Persistência dos dados, incluindo scripts de criação ou migração do banco e dados fictícios para demonstração.
- Testes automatizados referentes aos requisitos implementados.
- Tratamento de erros e validações das regras de negócio.
- Arquivos de configuração necessários para instalar, executar e testar o sistema, sem incluir senhas ou credenciais reais.

Os requisitos entregues nas releases anteriores deverão continuar funcionando.

Documentação técnica no README

O **README.md principal** deverá apresentar o projeto e conter as seguintes informações:

(1) Apresentação e escopo do sistema

Descrição do ClinicalStudyPB, perfis de usuários, funcionalidades principais e identificação dos seis integrantes da equipe.

(2) Arquitetura utilizada

Descrição da organização do sistema e das responsabilidades dos componentes, acompanhada dos diagramas de:

- Casos de uso.
- Classes ou modelo de domínio.
- Sequência dos principais fluxos.
- Arquitetura, mostrando a comunicação entre web, mobile, backend e banco de dados.

Os diagramas poderão ser inseridos como imagens ou apresentados em Mermaid, quando compatíveis com o GitHub.

(3) Funcionalidades desenvolvidas

Relação dos requisitos implementados, identificados pelos códigos RF, com descrição do comportamento e **capturas das interfaces web e mobile** demonstrando exemplos de uso. Incluir situações de sucesso e de erro ou validação.

(4) Stack tecnológico

Descrição das tecnologias e das **versões utilizadas** no backend, web, mobile, banco de dados, testes e ferramentas de execução. Informar também os pré-requisitos do ambiente.

(5) Instalação e execução

Instruções completas para executar o sistema, incluindo configuração do banco, inicialização do backend e da aplicação web, execução do aplicativo mobile em dispositivo ou emulador e credenciais fictícias para demonstração dos diferentes perfis.

(6) Testes e cobertura

Descrição dos testes implementados, resultados e evidências de cobertura.

Apresentar relatório de testes.

Demonstrar o atendimento à cobertura mínima de **80% das linhas do código de negócio do backend**.

(7) Documentação da API

Disponibilizar a documentação da API no Swagger UI.

Documentação de cada release

Para preservar o histórico das entregas, cada equipe deverá manter uma pasta **releases**, com uma subpasta para cada release. Cada subpasta deverá conter um **README.md próprio**, com o registro daquela entrega e links para suas evidências.

Entrega	Documento
Release 1	releases/release1/README.md
Release 2	releases/release2/README.md
Release 3	releases/release3/README.md

O README de cada release deverá apresentar:

- Objetivo da entrega e sprints abrangidas.
- Requisitos previstos, concluídos, parcialmente implementados e não implementados, com justificativas.
- Funcionalidades entregues e capturas das interfaces.
- Alterações na arquitetura e no stack, quando houver.
- Resultados dos testes e cobertura daquela versão.
- Problemas conhecidos e pendências.

O README principal deverá conter links para as três releases. Imagens, diagramas e evidências poderão ser armazenados em subpastas e incorporados aos documentos por links relativos.

Gerenciamento do processo de desenvolvimento

No README de cada release, incluir uma seção **Processo de desenvolvimento**, contendo, **para cada sprint**:

(1) Backlog e tarefas

Apresentar o objetivo da sprint, os itens de trabalho (PBIs) selecionados, as tarefas.

(2) Organização da equipe

Descrever os papéis e as responsabilidades dos seis integrantes, indicando suas contribuições na sprint. As atividades de testes, documentação e revisão de código deverão ser compartilhadas.

(3) Burndown

Incluir o gráfico de burndown de cada sprint, gerado pelo Azure DevOps, acompanhado de uma breve análise do planejado e do realizado.

(4) Revisão

Registrar o que foi entregue, os impedimentos encontrados e as ações de melhoria definidas para a próxima sprint.

Identificação e apresentação da entrega

Ao finalizar cada release, a equipe deverá:

- Criar uma **tag no GitHub**, identificada como release1, release2 ou release3, apontando para a versão entregue.
- Criar uma **GitHub Release** associada à tag, com um resumo e link para o README daquela entrega.
- Apresentar o sistema em funcionamento e demonstrar os requisitos implementados, as interfaces e os testes.

A avaliação utilizará a versão identificada pela tag, preservando o código e a documentação existentes no momento da entrega.

Documentação adicional da Release 3

Após a troca de projetos, a equipe receptora deverá registrar:

- Versão recebida e resultado da instalação e execução.
- Análise da arquitetura, do código e dos testes existentes.
- Plano de manutenção e requisitos surpresa.
- Alterações e refatorações realizadas.
- Testes de regressão e atualização das instruções de execução.